

music4all.ru — Документация

Содержание

1. [Быстрый старт](#)
 2. [Учётные данные по умолчанию](#)
 3. [Функциональность приложения](#)
 4. [Развёртывание на хостинге](#)
 5. [Миграция на Firebase](#)
 6. [Миграция на Supabase](#)
 7. [Безопасность в продакшене](#)
 8. [Часто задаваемые вопросы](#)
-

Быстрый старт

Приложение представляет собой **один HTML-файл** (`index.html`), не требующий сервера или базы данных. Просто откройте файл в браузере или загрузите на любой хостинг.

Откройте `index.html` в браузере → готово!

Учётные данные по умолчанию

Роль	Телефон	Пароль
Администратор	+79991234567	admin123
Ученик 1 (Иван Петров)	+79161234567	student1
Ученик 2 (Мария Сидорова)	+79261234568	student2

Важно: Смените пароль администратора сразу после первого входа! Для этого удалите ключ `m4a_initialized` в `localStorage` и обновите данные в функции `initSampleData()`.

Функциональность приложения

Для администратора

- Вход через отдельную страницу (ссылка «Вход для администратора» на главной)
- **Вкладка «Ученики»:** просмотр, добавление, редактирование, удаление учеников
- **Вкладка «Уроки»:** добавление уроков с видео (YouTube) и PDF-материалами
- **Вкладка «Песни»:** добавление песен с аккордами/табулатурами и YouTube-ссылкой
- **Вкладка «Назначения»:** выбор ученика и назначение/снятие уроков и песен

Для ученика

- Вход по номеру телефона и паролю
- **«Мои уроки»:** список назначенных уроков, встроенный YouTube-плеер, ссылка на PDF

- «**Мои песни**»: список назначенных песен, аккорды/табулатуры, YouTube-видео
- **Прогресс**: отметка уроков/песен как выполненных, визуальный прогресс-бар

Хранение данных

Все данные хранятся в `localStorage` браузера под ключами:

Ключ	Содержимое
<code>m4a_users</code>	Пользователи (ученики и администратор)
<code>m4a_lessons</code>	Уроки
<code>m4a_songs</code>	Песни
<code>m4a_assignments</code>	Назначения (ученик → урок/песня)
<code>m4a_progress</code>	Прогресс учеников
<code>m4a_session</code>	Текущая сессия
<code>m4a_theme</code>	Тема (dark/light)
<code>m4a_initialized</code>	Флаг первичной инициализации

Развёртывание на хостинге

Вариант 1: GitHub Pages (бесплатно)

```
# 1. Создайте репозиторий на GitHub
# 2. Загрузите index.html в корень репозитория
# 3. Перейдите: Settings → Pages → Source: Deploy from branch → main → /
(root)
# 4. Ваш сайт будет доступен по адресу: https://username.github.io/repo-
name/
```

Вариант 2: Netlify (бесплатно, рекомендуется)

```
# Способ 1: Drag & Drop
# Перейдите на https://app.netlify.com/drop
# Перетащите папку с index.html → сайт готов за 30 секунд!

# Способ 2: через CLI
npm install -g netlify-cli
netlify deploy --prod --dir=.
```

Вариант 3: Vercel (бесплатно)

```
npm install -g vercel
cd /папка/с/файлом
vercel --prod
```

Вариант 4: Timeweb / Beget / REG.RU (платный хостинг)

1. Подключитесь к хостингу через FTP (FileZilla или аналог)
2. Загрузите index.html в папку public_html или www
3. Если домен music4all.ru уже привязан – сайт сразу доступен
4. Для HTTPS: включите SSL-сертификат в панели управления хостингом

Вариант 5: VPS сервер (nginx)

```
# /etc/nginx/sites-available/music4all.ru
server {
    listen 80;
    server_name music4all.ru www.music4all.ru;
    root /var/www/music4all;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

```
# Загрузка файла
scp index.html user@your-server:/var/www/music4all/
sudo nginx -t && sudo systemctl reload nginx
```

Миграция на Firebase

Шаг 1: Создание проекта Firebase

1. Перейдите на <https://console.firebase.google.com/>
2. Нажмите «Добавить проект» → введите название
3. Включите Google Analytics (опционально)
4. В разделе «Создание» → «Firestore Database» → «Создать базу данных»
5. Выберите режим «Тестовый» (для разработки)
6. В разделе «Создание» → «Authentication» → «Начать»
7. Включите провайдер «Телефон» (или создайте кастомную аутентификацию)

Шаг 2: Получение конфигурации

```
// В консоли Firebase: Настройки проекта → Ваши приложения → Веб
const firebaseConfig = {
  apiKey: "AIzaSy...",
  authDomain: "your-project.firebaseio.com",
  projectId: "your-project",
  storageBucket: "your-project.appspot.com",
  messagingSenderId: "123456789",
  appId: "1:123456789:web:abc123"
};
```

Шаг 3: Замена слоя данных

Добавьте в `<head>` файла:

```

<script type="module">
  import { initializeApp } from
  "https://www.gstatic.com/firebasejs/10.7.0/firebase-app.js";
  import { getFirestore, collection, getDocs, addDoc, updateDoc, deleteDoc,
  doc, query, where }
  from "https://www.gstatic.com/firebasejs/10.7.0/firebase-firestore.js";

  const app = initializeApp(firebaseConfig);
  const db_firebase = getFirestore(app);

  // Замените функции db.getUsers(), db.getLessons() и т.д.:

  // Получить всех пользователей
  async function getUsers() {
    const snap = await getDocs(collection(db_firebase, 'users'));
    return snap.docs.map(d => ({ id: d.id, ...d.data() }));
  }

  // Добавить пользователя
  async function addUser(userData) {
    return await addDoc(collection(db_firebase, 'users'), userData);
  }

  // Обновить пользователя
  async function updateUser(id, data) {
    await updateDoc(doc(db_firebase, 'users', id), data);
  }

  // Удалить пользователя
  async function deleteUser(id) {
    await deleteDoc(doc(db_firebase, 'users', id));
  }

  // Аналогично для lessons, songs, assignments, progress
</script>

```

Шаг 4: Структура коллекций Firestore

Firestore

├─ users/

| └─ {userId}: { role, phone, passwordHash, name, createdAt }

├─ lessons/

| └─ {lessonId}: { title, description, videoUrl, pdfUrl, createdAt }

├─ songs/

| └─ {songId}: { title, artist, difficulty, youtubeUrl, tabsChords,
createdAt }

├─ assignments/

| └─ {assignId}: { studentId, type, contentId, assignedAt }

└─ progress/

└─ {progressId}: { studentId, type, contentId, completed, updatedAt }

Шаг 5: Правила безопасности Firestore

```
// firestore.rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Только аутентифицированные пользователи
    match /{document=**} {
      allow read, write: if request.auth != null;
    }

    // Ученики могут читать только свои данные
    match /assignments/{id} {
      allow read: if request.auth.uid == resource.data.studentId
        || isAdmin();
    }

    match /progress/{id} {
      allow read, write: if request.auth.uid == resource.data.studentId
        || isAdmin();
    }

    function isAdmin() {
      return
get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role ==
'admin';
    }
  }
}
```

Миграция на Supabase

Шаг 1: Создание проекта

1. Перейдите на <https://supabase.com/>
2. Нажмите «New project»
3. Введите название и пароль базы данных

Шаг 2: Создание таблиц (SQL)

```
-- Пользователи
CREATE TABLE users (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  role TEXT NOT NULL DEFAULT 'student',
  phone TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  name TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Уроки
CREATE TABLE lessons (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  title TEXT NOT NULL,
  description TEXT,
  video_url TEXT,
  pdf_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Песни
CREATE TABLE songs (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  title TEXT NOT NULL,
  artist TEXT NOT NULL,
  difficulty TEXT DEFAULT 'beginner',
  youtube_url TEXT,
  tabs_chords TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Назначения
CREATE TABLE assignments (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  student_id UUID REFERENCES users(id) ON DELETE CASCADE,
  type TEXT NOT NULL, -- 'lesson' or 'song'
  content_id UUID NOT NULL,
  assigned_at TIMESTAMPTZ DEFAULT NOW()
);

-- Прогресс
CREATE TABLE progress (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
```

```
student_id UUID REFERENCES users(id) ON DELETE CASCADE,  
type TEXT NOT NULL,  
content_id UUID NOT NULL,  
completed BOOLEAN DEFAULT FALSE,  
updated_at TIMESTAMPTZ DEFAULT NOW(),  
UNIQUE(student_id, type, content_id)  
);
```

Шаг 3: Подключение Supabase JS SDK

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>  
<script>  
  const { createClient } = supabase;  
  const supabaseClient = createClient(  
    'https://your-project.supabase.co',  
    'your-anon-key'  
  );  
  
  // Пример: получить всех учеников  
  async function getStudents() {  
    const { data, error } = await supabaseClient  
      .from('users')  
      .select('*')  
      .eq('role', 'student');  
    return data;  
  }  
  
  // Пример: добавить урок  
  async function addLesson(lesson) {  
    const { data, error } = await supabaseClient  
      .from('lessons')  
      .insert([lesson]);  
    return data;  
  }  
</script>
```

Безопасность в продакшене

Внимание! Текущая реализация использует `base64` для хеширования паролей — это **только для демонстрации**. В продакшене обязательно используйте надёжное хеширование.

Рекомендации для продакшена:

1. **Хеширование паролей:** Используйте `bcrypt` на сервере (Node.js/Python) или встроенную аутентификацию Firebase/Supabase.
2. **Сессии:** Замените `localStorage`-сессии на JWT-токены с коротким сроком жизни.
3. **HTTPS:** Всегда используйте SSL/TLS сертификат (Let's Encrypt — бесплатно).
4. **Переменные окружения:** Никогда не храните API-ключи в клиентском коде. Используйте серверные функции (Firebase Functions, Supabase Edge Functions).
5. **Row Level Security:** В Supabase включите RLS для всех таблиц.
6. **Смена пароля администратора:** Перед публикацией обязательно измените `admin123` на надёжный пароль.

Часто задаваемые вопросы

Q: Как сбросить все данные?

```
// Откройте консоль браузера (F12) и выполните:  
Object.keys(localStorage).filter(k => k.startsWith('m4a_')).forEach(k =>  
  localStorage.removeItem(k));  
location.reload();
```

Q: Как изменить пароль администратора? Откройте `index.html`, найдите функцию `initSampleData()` и измените строку:

```
passwordHash: hashPassword('+79991234567', 'admin123'),  
// Замените на:  
passwordHash: hashPassword('+79991234567', 'ВАШ_НОВЫЙ_ПАРОЛЬ'),
```

Затем сбросьте данные через консоль (см. выше) и перезагрузите страницу.

Q: Как добавить нового администратора? В функции `initSampleData()` добавьте ещё один объект с `role: 'admin'`.

Q: Данные сохраняются только в одном браузере? Да, `localStorage` привязан к браузеру и устройству. Для синхронизации между устройствами необходима база данных (Firebase/Supabase).

Q: Как встроить PDF-материалы? Загрузите PDF на Google Drive, Яндекс.Диск или любое облако, получите публичную ссылку и вставьте её в поле «Ссылка на PDF материалы» при создании урока.

Q: Поддерживаются ли другие видеохостинги кроме YouTube? Текущая реализация оптимизирована под YouTube (автоматическое извлечение ID). Для Vimeo или других платформ можно вставить прямую ссылку на видео — оно откроется в новой вкладке.